

Getting Started With Yahboom's Dofbot

Kimberly Oakes, email: oakesk2@rpi.edu

August 2022

1 Introduction

This document was written to introduce the basics of using Yahboom's Dofbot. This guide will help navigate some of the capabilities of the arm as well as available online resources.

This is meant to be both a learning guide and a resource for identifying problems and troubleshooting. For this reason, this document will be an adaptive one. Any suggestions for additions, improvements, or changes are appreciated. In addition, if you encounter any problems while operating the Dofbot that are not already listed, please let the author know.

2 Dofbot Overview

Dofbot is a desktop sized robot arm with six servos, a wrist mounted camera, a [Nvidia Jetson Nano](#), and an [expansion board](#) for advanced development. Programming can be done in [ROS](#) and [Python](#). A visual is provided in figure 1.



Figure 1: [Dofbot](#)

3 Yahboom Online Resources

Tutorials can be found at http://www.yahboom.net/study/Dofbot-Jetson_nano. The content is accessible by the menu on the left side. It is important to note that not all sections apply. It is possible to buy the arm without the Jetson Nano. For this reason, there are categories for setting up other boards and installing firmware. This has already been done. This document will direct you toward some specific sections of this site. If you are uncertain of the applicability of other sections, please ask.

Yahboom also has a GitHub repository that includes PDF versions of the tutorials from the previously mentioned site. It also contains content from the Downloads section of the previous site. The repository can be accessed here: https://github.com/YahboomTechnology/dofbot-jetson_nano.

The code that is run in the tutorials is on Dofbot's operating system. If you wish to look at it on your own, it can be downloaded from this [link](#). This is a Google Drive link to a zip folder. After unzipping, you should see the folders and code as they will appear on Dofbot's operating system.

4 Some Important Notes

- Collision avoidance is not built-in by default. **USE CAUTION** when moving joints. Self-collision may occur.
- During early testing, the servo for the gripper started feeling warm. Keep this in mind and shut down if it starts getting hot to the touch.
- The joint range is 0-180 degrees for each joint (except joint 5). The python module for controlling the servos may let you specify a number outside this range. However, if the joint angle is outside $[0, 180]$ degrees, the read function will return nil. Therefore, do not command the joints outside $[0, 180]$ degrees.
- Use caution when commanding near 0 or 180 degrees. Gravitational forces may result in an actual angle $\neq 0$ or $\neq 180$. This would make the read angle function return nil.
- Fast movements may produce forces which can dislodge the suction cups
- If running code or the app and the arm is not moving, turn it off and back on.

5 Starting Up

- Ensure robot is plugged in.
- If accessing Dofbot's internal computer directly, connect mouse, keyboard, and monitor.

- If using camera, remove cap. **Replace camera cap when done using arm.**
- Power switch is located on the bottom board, near the antennas.
- When Dofbot is ready, you will hear a buzzer sound.

6 Utilizing The App

The app is a great way to get familiar with the joint movements and ranges. The app is available both for iOS and Android. The app is named YahboomRobot or you can use the QR code (for Android) on the [tutorials page](#). This page is accessed by the left-hand menu, under *1. First Trial & APP Control*. A visual of this page is provided in figure 2.

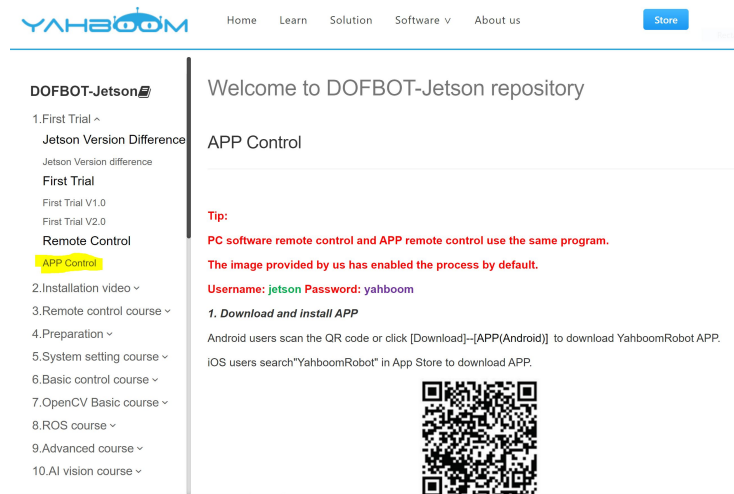


Figure 2: Location of QR code for app download on Android.

6.1 Connecting With The App

Be sure your phone is connected to the duckietown Wi-Fi. Open the app. You should see the screen shown in figure 4. Click on *IP Connect*. This should bring up the screen seen in figure 5. Input the IP address visible on the OLED board (as seen in figure 3) on the Dofbot and click *Connect*.



Figure 3: OLED board showing Dofbot IP address.

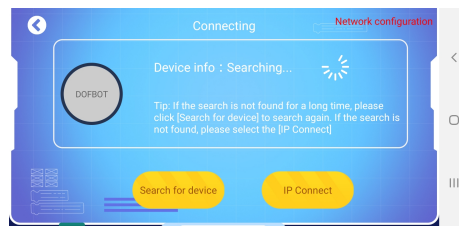


Figure 4: App start screen.

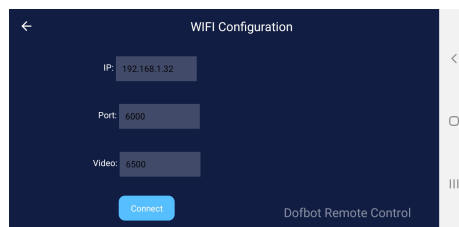


Figure 5: IP address entrance screen for Wi-Fi connection.

6.2 Using The App

There are multiple functions available on the app. These are described on the tutorials page under *3. Remote Control Course : 3.2 APP Control* as seen in figure 6. Note that color calibration has not been accurately performed so the color based functions may not work as intended.

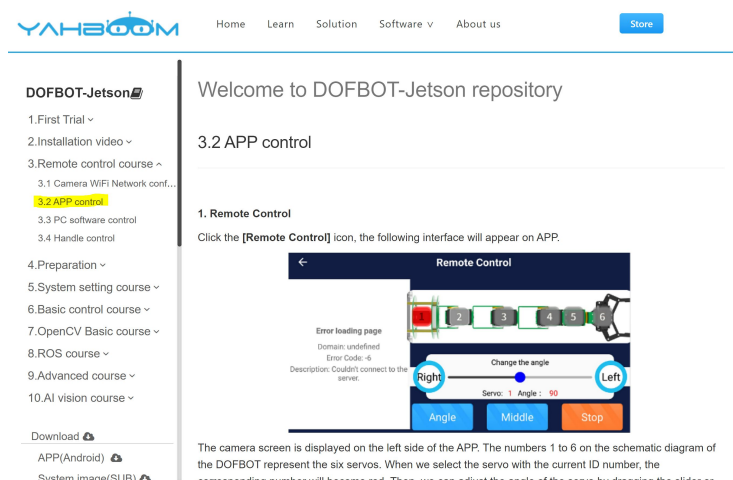


Figure 6: Location of descriptions of app functions.

The function we are interested in is the remote control as seen in figure 7. The left side shows the video from the camera. On the top right, the joint to move can be selected. The up/down buttons allow you to increment the joint angle. The sliders can be used to make larger movements. **USE CAUTION** when using the sliders since self-collision may occur. The *Stop* button in the bottom right is used to turn Dofbot control on and off. The *Middle* button centers all servos. This results in the arm pointing straight up.

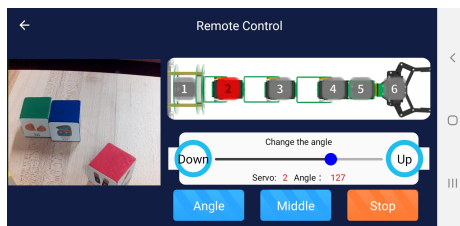


Figure 7: Remote control screen from the app.

You should use the app to get familiar with the ranges and movement directions of each joint. Make notes about the positive/negative movement directions of each joint. Also note 0° , 45° , 90° , 135° , and 180° positions. This will be useful information to know when we start implementing some simple code of our own.

After exploring some of the provided code, an exercise will be presented. In the exercise, you will create a script to control individual joint movements using a keyboard. Again, there will be know self-collision avoidance. It is important to know the ranges of each joint beforehand to ensure safe operation.

7 Dofbot Operating System

The Dofbot is equipped with Nvidia's Jetson Nano, a small computer with the computational capacity to implement complex machine learning algorithms. The operating system is Linux4Tegra, based on Ubuntu 18.04. The operating system can be directly accessed by plugging in an HDMI monitor, USB mouse, and USB keyboard into the Jetson Nano.

8 Getting Started With JupyterLab

[JupyterLab](#) is a web-based application designed for ease of use and promotion of open source coding. The application can be used as a text editor, terminal, notebook editor, and more. Figure 8 shows the main screen. The language we will be using with JupyterLab is Python. Note that you must use JupyterLab or Jupyter Notebook to read files of type *.ipynb.

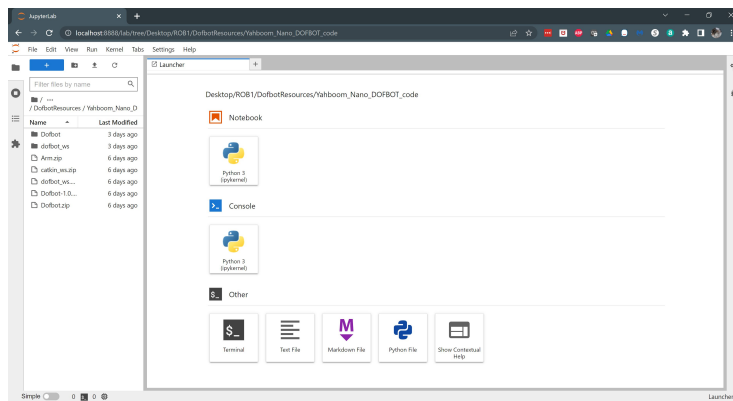


Figure 8: JupyterLab main screen.

8.1 Connecting To JupyterLab

8.1.1 On The Dofbot

To connect to JupyterLab on the Dofbot, first open the browser. The Chromium web browser is on the main screen with an icon that looks like a blue-scale version of Chrome. In the address bar type: *localhost:8888*. You may be prompted to

input a password when opening the browser. This password is *yahboom*. You should see the main JupyterLab screen.

8.1.2 To Dofbot From A Laptop

You can access JupyterLab remotely from a laptop. First, make sure that you are connected to the duckietown Wi-Fi. Then open a web browser. In the address bar type: *IP*:8888, where *IP* is the IP address of the Dofbot. The IP address is visible on the OLED screen on the Dofbot. You should see the main JupyterLab screen.

8.1.3 Laptop Without Dofbot

The following instructions are based on a Windows operating system. If you have another, the procedure may differ.

If you wish to create and test notebook files on your own laptop, you must install JupyterLab locally. First, if you do not have the latest version of Python on your laptop, download it at <https://www.python.org/downloads/>. Be sure to click add Python 3.10 to PATH. This will allow you to access Python using the terminal.

Not all Python modules are installed by default. A common module that is used is *numpy*. This module is used for forming and manipulating matrices. To install *numpy*, open a terminal (open command prompt using search bar). Type *pip install numpy*. Hit enter and *numpy* will be installed.

In the search bar, type command prompt. Launch the command prompt app. This opens a terminal where we can install JupyterLab. Type: *pip install jupyterlab*. Allow *pip* to finish installing JupyterLab. When complete, type *jupyterlab* to launch JupyterLab in your default browser. You must leave the terminal open while using JupyterLab. Close the browser and terminal when you are done. To connect again, open the command prompt and type *jupyterlab* as done previously. When JupyterLab is running, the terminal looks like figure 9.

8.2 First Files To Look At On Dofbot

A good way to learn some of the basic modules and format for Notebooks is to look at existing files. Navigate to 6. *Basic control course* at http://www.yahboom.net/study/Dofbot-Jetson_nano as seen in figure 10. Specifically focus on 6.3, 6.4, and 6.5. (Note: if you follow 6.1, skip to Intro to API. You do not need to install any source code.)

8.3 Navigating JupyterLab

Click on *File* in the top left. Then select *New* as seen in figure 11. You can open a new terminal, create a new Python file, or create a new Notebook. In this section we will focus on Notebooks.

```
Command Prompt - jupyter-lab
Microsoft Windows [Version 10.0.19043.1826]
(c) Microsoft Corporation. All rights reserved.

C:\Users\oakes>jupyter-lab
[I 2022-08-09 08:51:19.830 ServerApp] jupyterlab | extension was successfully linked.
[I 2022-08-09 08:51:19.846 ServerApp] nbclassic | extension was successfully linked.
[I 2022-08-09 08:51:20.489 ServerApp] notebook_shim | extension was successfully linked.
[I 2022-08-09 08:51:20.552 ServerApp] notebook_shim | extension was successfully loaded.
[I 2022-08-09 08:51:20.555 LabApp] JupyterLab extension loaded from C:\Users\oakes\AppData\Local\Programs\Python\Python3
10\lib\site-packages\jupyterlab
[I 2022-08-09 08:51:20.555 LabApp] JupyterLab application directory is C:\Users\oakes\AppData\Local\Programs\Python\Pyth
on310\share\jupyterlab
[I 2022-08-09 08:51:20.562 ServerApp] jupyterlab | extension was successfully loaded.
[I 2022-08-09 08:51:20.575 ServerApp] nbclassic | extension was successfully loaded.
[I 2022-08-09 08:51:20.577 ServerApp] Serving notebooks from local directory: C:\Users\oakes
[I 2022-08-09 08:51:20.577 ServerApp] Jupyter Server 1.8.1 is running at:
[I 2022-08-09 08:51:20.578 ServerApp] http://localhost:8888/lab?token=9d751f8872fd0597bfcdb239994b43a6299b3334f2d68d9
or http://127.0.0.1:8888/lab?token=9d751f8872fd0597bfcdb239994b43a6299b3334f2d68d9
[I 2022-08-09 08:51:20.578 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
[C 2022-08-09 08:51:20.698 ServerApp]

To access the server, open this file in a browser:
file:///C:/Users/oakes/AppData/Roaming/jupyter/runtime/jpservice-28988-open.html
Or copy and paste one of these URLs:
https://localhost:8888/lab?token=9d751f8872fd0597bfcdb239994b43a6299b3334f2d68d9
or http://127.0.0.1:8888/lab?token=9d751f8872fd0597bfcdb239994b43a6299b3334f2d68d9
[W 2022-08-09 08:51:25.635 LabApp] Could not determine jupyterlab build status without nodejs
```

Figure 9: Opening JupyterLab in a terminal.

8.3.1 Creating And Running A Notebook

When you select *File:New:Notebook*, the screen prompts you to start a kernel as seen in figure 12. Click *Select* to connect the new file to a Python3 kernel.

Some important icons from the top ribbon include:

- +: adds a new cell
- play: runs the currently selected cell
- double play: runs the full file
- stop: stops the file execution

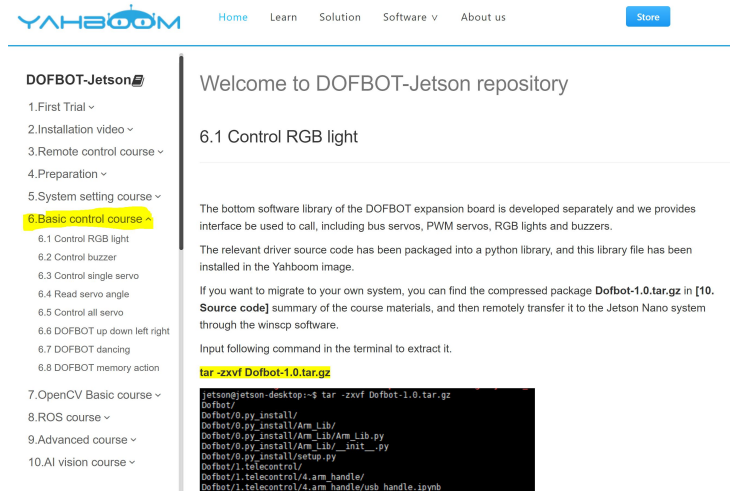


Figure 10: Module introduction files.

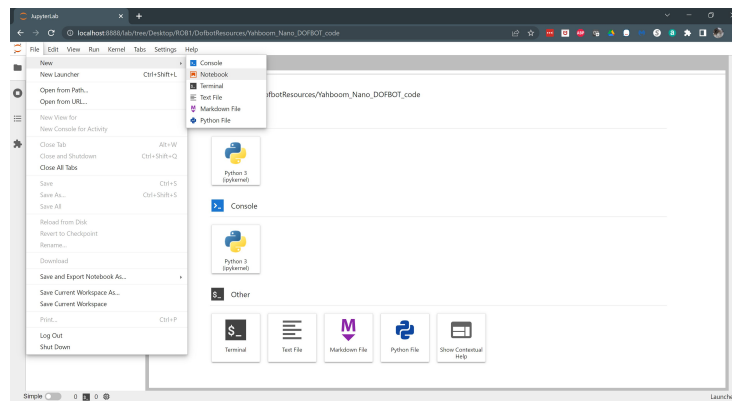


Figure 11: JupyterLab capabilities.

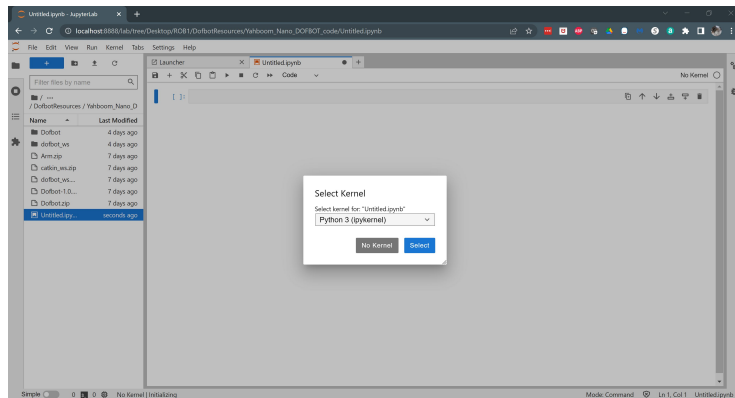


Figure 12: Selecting a kernel for new JupyterLab Notebook.

9 First Coding Exercises

To get you looking more closely at the code and starting to understand how it works, you will fill in the blanks for two provided files. On [Piazza](#), under Project in the Resources tab, download *FillIn_keyboardJointMove_noDofbot.ipynb*. This file does not require the use of Dofbot, so it can be run on your own laptop. Open the downloaded file in JupyterLab. Look through the code and fill in any spots where you see _____. After filling in the blanks, run the file by clicking the double play button. Prompts should show up at the bottom of the file. Enter the joint number as directed and press Enter. Test the code robustness. What happens if you enter an invalid joint number or angle?

Once you have successfully modified this file, modify the follow-up file. Download *FillIn_keyboardJointMove_withDofbot.ipynb* from Piazza. This file is like the previous. We input the joint number and angle to move a single joint using the keyboard. The difference is this file will actually move the corresponding joint on the Dofbot. This file does require connection to Dofbot to run. You can modify the code on your own, then test on the Dofbot. You can move files from your laptop to the Dofbot operating system using a USB drive. Please create a folder on the desktop for your team and keep all of your files in that folder. The Dofbot's are shared, so please be respectful of other teams' code.

Be careful when running your first file using the Dofbot. As previously mentioned, self-collision can occur so it is best to use small incremental movements at first.

A Some Useful Python Commands And Modules

Some important things to note about Python that may be different from other programming languages you have used:

- Indexing starts at 0.
- Indentation is important.

The following commands and modules are frequently used. You will see many of them in the sample codes and should look up the details of their use.

- `import`: used to add a module to the current space
- `numpy`: module used for arrays and matrices
- `cv2`: openCV module used for camera and image operations
- `time`: module used for pausing and timing code executions
- `if, elif, else`: used for if/else statements
- `while`: used for while loops
- `break, continue`: useful for exiting, skipping lines in loop

B Python Dofbot Module Notes

- Arm.Lib: The main module used to control the Dofbot
 - Arm.Device: The main submodule of Arm.Lib
 - Some important functions of Arm.Device
 - * Arm_serial_servo_write(id,angle,time): Move joint number id to angle degrees in time milliseconds. Specifying time=0 will have the joints move as fast as possible.
 - * Arm_serial_servo_write6(a1,a2,a3,a4,a5,a6,time): Move joint 1 to a1 degrees, joint 2 to a2 degrees, etc. Move joints in time milliseconds.
 - * Arm_serial_servo_read(id): Returns the position of joint number id in degrees.

C Some Useful Terminal Commands

Below is listed a couple of commands that may be useful when starting using a terminal.

- ls: list the folders and files in the current directory
- cd: change the current directory
- python: starts a Python environment
 - help: used in the Python environment to get more information about a module. Example: help(time)
 - q: exits the help display
 - exit(): exits the Python environment